

INTERNAL — PREPARED FOR EXECUTIVE LEADERSHIP

The vBank Cyber Range

*A self-hosted, live-fire application security training platform
— built to turn "we did awareness training" into "we watched
our people break a bank, safely."*

PREPARED FOR	PREPARED BY	DATE	CLASSIFICATION
CISO & Executive Leadership	Security Engineering	7 July 2026	Internal — Confidential
PLATFORM VERSION			
1.3 (Production)			

CONTENTS

Executive summary	01	Why this exists	02
Platform blueprint	03	The player experience	04
The administrator experience	05	Types of challenges	06
The 20-challenge catalog	07	Connectivity & infrastructure range	08
Containment & safety model	09	Scoring & progression	10
Technology stack	11	Scalability & deployment	12
Recommendations	13	Appendix — OWASP mapping	14

— 01 — EXECUTIVE SUMMARY

A private banking environment our own people are allowed to break

vBank is a fully-functional, intentionally vulnerable online bank that we built and control end-to-end. Every employee gets their own private, disposable copy of it — and a browser-based hacking workstation to attack it with. Nothing here touches production; everything here builds real skill.

The platform runs 20 hands-on security challenges modelled directly on the vulnerability classes behind real-world banking breaches — injection, broken access control, weak cryptography, authentication failures, insecure design, and server-side request forgery. Rather than a slide deck, participants get a working attack surface, a working attacker toolkit, and a scoreboard. They learn what an SQL injection actually does by using one to drain an account; they learn why signed tokens matter by forging one.

20 Hands-on challenges	6,075 Total points in play	7 OWASP-mapped vulnerability classes	1-click Per-user isolated target & toolkit
0 Software installed on player laptops	4-node Tested horizontal scale-out		

Everything is self-hosted on infrastructure we own, runs on a single machine or a small cluster, and requires no installed software on a player's laptop beyond a web browser — deliberately compatible with locked-down corporate devices. Every player is fully isolated in their own Docker network; there is no shared blast radius, and every environment self-destructs on a timer.

02 — WHY THIS EXISTS

The gap between "aware" and "able"

Conventional security-awareness training tells people what a vulnerability is. It rarely lets them cause one, watch it fail, and fix their mental model. The vBank range closes that gap for the audiences that most need it:

Engineering & QA

Developers who have personally exploited a SQL injection or an IDOR write more defensive code — and review pull requests differently — than developers who have only read about them.

Internal security & red team

A repeatable, resettable range to onboard new analysts, rehearse tooling (Burp Suite, sqlmap, jwt_tool), and calibrate skill level before they touch anything live.

Risk & compliance

A concrete, demonstrable artefact of proactive security investment — useful for board reporting, audit evidence, and vendor/partner due-diligence conversations.

Culture

A scoreboard and a shared, slightly irreverent banking parody turns "mandatory training" into something people actually want to spend an afternoon on.

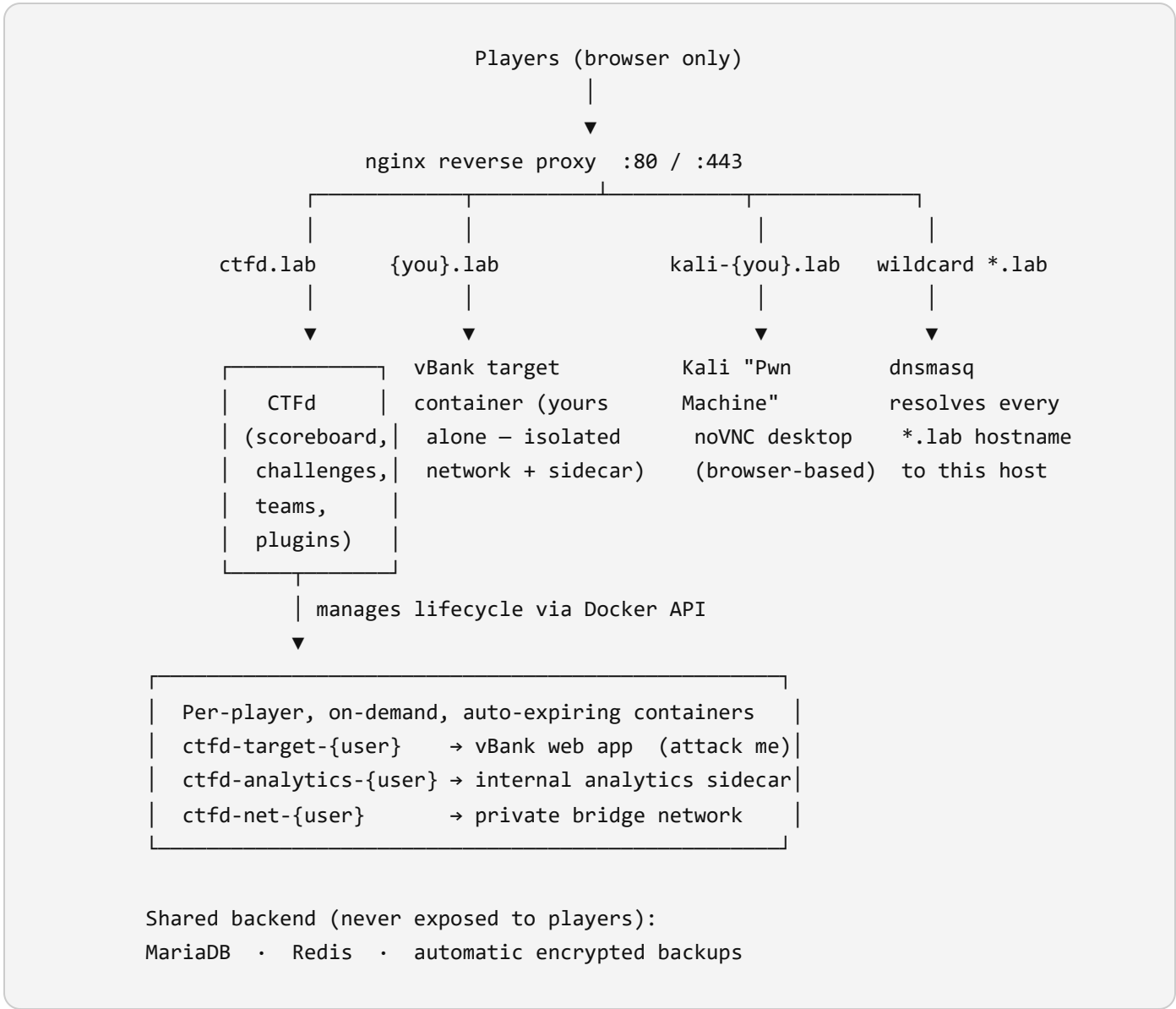
THE BUSINESS CASE IN ONE LINE

Every vulnerability class in this range has a documented real-world banking-sector breach behind it. This is not a generic CTF re-skinned with a bank logo — the attack surface, the data model (accounts, transfers, statements, staff portal, payroll), and the failure modes were purpose-built to mirror how these bugs actually show up in financial services applications.

03 — PLATFORM BLUEPRINT

How it's built

The platform is a Docker Compose stack fronted by a single reverse proxy and a wildcard DNS server. CTFd (an open-source competition engine, extensively customised) is the front door for every player; a purpose-built plugin provisions and tears down each player's private infrastructure on demand.



One reverse proxy and one DNS server route every player to their own private stack; the database and cache sit on an internal-only network with no route to the internet or to players.

The three moving parts

- **CTFd** — the scoreboard, challenge descriptions, hints, teams, and admin console. Heavily extended with in-house plugins for target provisioning, live notifications, and a cyberpunk-themed UI.
- **vBank** — the target. A realistic Flask-based online banking application (login, dashboard, transfers, statements, a staff portal, payroll, support tooling) with twenty deliberately engineered vulnerabilities.
- **The Pwn Machine** — a full Kali Linux desktop, delivered to the player's browser over noVNC, pre-loaded with the industry-standard attacker toolkit (Burp Suite, sqlmap, hydra, hashcat, nmap, jwt_tool, and more).

— 04 — THE PLAYER EXPERIENCE

What it feels like to be a participant

No installation, no VPN client, no VM to configure. A player goes from a cold start to actively attacking a bank in under two minutes.

1 Register and land on the challenge board

One account per person. Every person gets full scoring and progress on their own — an optional `Team Hub` lets colleagues group up for collaboration, but nobody's score depends on having a team.

2 Launch Target

One click on the challenges page spins up a private vBank instance plus its analytics sidecar, on an isolated network, reachable at the player's own URL. Live status and a countdown timer are shown on the page; the session auto-extends while the player stays active and expires automatically otherwise.

3 Launch the Pwn Machine (optional)

A second click opens a full Kali desktop in a browser tab — no local install, works from a locked-down corporate laptop, includes Burp Suite, sqlmap, hydra, hashcat, nmap, jwt_tool, wordlists, and Python security libraries out of the box.

4 Work the 20 challenges

Progressive difficulty from "read the page source" to a multi-step SSRF-into-deserialization chain. Each challenge ships with graduated hints (at a small point cost) so nobody gets permanently stuck.

5 Submit flags, watch the board move

Flags are unique per player (HMAC-derived from their username), so scoreboard positions reflect genuine, individual work — copy-pasting someone else's flag doesn't work.

6 Get notified when teammates score

Optional-team players see a live toast notification when a teammate solves a challenge — small dopamine hit, real engagement driver.

— 05 — THE ADMINISTRATOR EXPERIENCE

What it takes to run an event

Designed so that one person — not a dedicated infrastructure team — can stand up, run, and tear down an event.

1 Deploy with one command

A single cross-platform launcher (`start.sh` / `start.ps1` / double-click `start.bat`) auto-detects the host's LAN IP, generates a self-signed CA and TLS certificates, builds every container image, and waits for a healthy stack. No manual Docker knowledge required to launch.

2 Distribute one certificate and one DNS setting

Players install a single CA certificate (downloadable from an in-app guide) and point their DNS at the host. From there, every `*.lab` hostname — the scoreboard, every player's bank, every player's Kali box — just resolves.

3 Watch the event live

The admin dashboard (`Plugin Menu` → `Target Machine`) shows every running container, owner, age, and remaining lifetime in real time, with one-click force-stop.

4 Approve optional team requests

Team captains approve or decline join requests from a native in-app notification — no admin involvement needed for day-to-day team management.

5 Data is protected automatically

Compressed database backups are taken automatically before any destructive action (a full scoreboard reset, a challenge deletion, or event end) and stored under `.data/CTFd/backups` . A destructive action is **cancelled** if its safety backup fails — data loss is opt-out only by explicit override, not the default failure mode.

6

Tear down or scale up

A single flag stops everything cleanly (`--down`); a documented HAProxy configuration scales the identical stack across up to four nodes with a shared database for larger cohorts.

CREDENTIAL & SECRETS HYGIENE

All secrets (admin password, session keys, JWT signing key, database password, per-event flag seed) live in a single untracked `.env` file, generated fresh per deployment and never committed to source control. Nothing in this document, or in the platform's own audit output, discloses a live credential.

— 06 — TYPES OF CHALLENGES

Seven vulnerability classes, mapped to how real breaches happen

The 20 challenges are not a random grab-bag — they were selected to cover the vulnerability classes that recur across real financial-services incident reports, each mapped to the OWASP Top 10 (2021). Difficulty rises across three tiers so a complete beginner and a seasoned pentester both find something calibrated to them.

6**Broken access control**

OWASP A01 • IDOR, mass data exposure, path traversal, open redirect, CSRF

5**Injection**

OWASP A03 • SQL injection (x2), stored XSS, command injection, template injection

3**Insecure design / business logic**

OWASP A04 • race conditions, sign-check bypass, mass assignment

2**Authentication failures**

OWASP A07 • forged JWTs, predictable reset tokens

2**Security misconfiguration**

OWASP A05 • secrets in source, XML external entities

1**Cryptographic failure**

OWASP A02 • ECB-mode encryption, hardcoded key

1**SSRF & insecure deserialization**

OWASP A10 / A08 • multi-step attack chain

Difficulty distribution

4 Basic · 11 Intermediate · 5 Advanced — a deliberately front-loaded curve so early wins build confidence before the multi-step chains later in the board.

07 — THE 20-CHALLENGE CATALOG

Every challenge, its real-world risk, and how it's solved

Full technical reproduction steps (Burp Suite request/response pairs, payloads, and remediation code) are maintained separately for the engineering and blue-team audience. Below is the executive view: what the bug is, why it matters commercially, and the shape of the attack path.

#	CHALLENGE	DIFFICULTY	VULNERABILITY CLASS	POINTS
1	The Intern's Mistake	Basic	Information disclosure	100
2	All Doors Open	Basic	SQL injection	150
3	The Ghost Investor	Basic	Broken access control (IDOR)	200
4	Flagged for Review	Basic	Broken access control	225
5	Unsigned Agreement	Intermediate	Authentication failure (JWT)	250
6	Feedback Loop	Intermediate	Stored XSS	275
7	Double Dip	Intermediate	Insecure design (race condition)	300
8	The Repeating Pattern	Intermediate	Cryptographic failure	325
9	In the Dark	Advanced	Blind SQL injection	350
10	Document Confession	Advanced	XXE / misconfiguration	375
11	No Token, No Problem	Intermediate	CSRF / broken access control	300
12	Money Glitch	Intermediate	Insecure design (business logic)	325

#	CHALLENGE	DIFFICULTY	VULNERABILITY CLASS	POINTS
13	Bait and Switch	Intermediate	Broken access control (open redirect)	275
14	Password Roulette	Intermediate	Authentication failure	350
15	The Vault Manifest	Intermediate	Broken access control (data exposure)	300
16	The Archivist	Intermediate	Broken access control (path traversal)	275
17	Internal Affairs	Advanced	Command injection	425
18	Display Name Injection	Advanced	Server-side template injection	450
19	The Fast Track	Intermediate	Insecure design (mass assignment)	325
20	God Mode	Advanced	SSRF chained to insecure deserialization	500

Tier 1 — Basic Foundations

01 The Intern's Mistake

Basic 100 pts

A hidden staff-portal URL and a secret sit in plain HTML and robots.txt — visible to anyone who looks at page source.

ATTACK PATH

1. View page source on the public site.
2. Check robots.txt for disallowed paths.
3. Follow the disclosed staff-portal URL and read its source.

BUSINESS RISK

Client-side comments and crawler directives are **not** access control. This is how internal admin panels get found by opportunistic scanners within hours of deployment.

02 All Doors Open

Basic 150 pts

The customer login concatenates the username straight into a SQL query — a classic authentication bypass.

ATTACK PATH

1. Open the login form and intercept the request.
2. Replace the username with ' OR 1=1 --.
3. The query returns true for every row; session is granted with zero valid credentials.

BUSINESS RISK

Full authentication bypass. One unparameterized query away from every customer account being accessible without a password.

03 The Ghost Investor

Basic 200 pts

The transaction-history API checks that you're logged in — but never that the account is yours.

ATTACK PATH

1. Log in as any customer.
2. Capture the statements request for your own account ID.
3. Swap the ID for another (the CEO's account) and replay it.

BUSINESS RISK

Textbook **Insecure Direct Object Reference**. In production this is unauthenticated financial data exposure for every customer, one sequential ID away.

04 Flagged for Review

Basic 225 pts

Compliance blocked a suspicious beneficiary in the UI — but the backend never re-checks the block when a request is crafted directly.

ATTACK PATH

1. Discover the blocked account's reference number via an unrelated IDOR.
2. Build a transfer-confirmation request targeting that reference directly.
3. Backend processes the transfer despite the compliance flag.

BUSINESS RISK

A UI-only control is not a control. This is the exact shape of a sanctions/AML bypass — **regulatory exposure**, not just a bug.

Tier 2 — Intermediate Escalation

05 Unsigned Agreement

Intermediate 250 pts

The staff API's JWT verification trusts the client-supplied algorithm — including none.

ATTACK PATH

1. Request a legitimate staff token.
2. Rewrite the header to `{"alg": "none"}` and the payload to `claim role: admin`.
3. Submit with an empty signature to the corporate vault endpoint.

BUSINESS RISK

Full privilege escalation to administrator with no valid credentials at all — a single misconfigured JWT library setting away from a total authorization bypass.

06 Feedback Loop

Intermediate 275 pts

Support-ticket text is rendered unescaped on the internal staff review page.

ATTACK PATH

1. Submit a ticket containing an `<img onerror>` payload.
2. Wait for staff to open the review queue.
3. The payload fires in the staff's browser and exfiltrates their session cookie.

BUSINESS RISK

Any customer can trigger code execution in a **staff member's authenticated session** — a direct pivot from low-trust to high-trust identity.

07 Double Dip

Intermediate 300 pts

Express Transfer checks the balance and deducts it in two separate steps, 300ms apart, with no locking.

ATTACK PATH

1. Capture one valid transfer request for the full balance.
2. Fire 20-30 copies of it in parallel (Burp's parallel sender or Turbo Intruder).
3. Multiple requests pass the same stale balance check before any deduction lands.

BUSINESS RISK

Double-spend / TOCTOU — the same fraud pattern behind real instant-payment abuse cases; a scaled attack can multiply free funds many times over.

08 The Repeating Pattern

Intermediate 325 pts

Receipts are "encrypted" with AES in ECB mode using a hardcoded, guessable key.

ATTACK PATH

1. Fetch an encrypted system receipt.
2. Recover the hinted 16-byte key.
3. Decrypt with AES-ECB; identical plaintext blocks are trivially visible and reversible.

BUSINESS RISK

ECB mode leaks structural patterns and offers zero integrity protection — this is a **cryptographic control that provides no real protection**, worse than having none because of the false assurance.

11 No Token, No Problem

Intermediate 300 pts

The bill-payment form has no CSRF token and no origin check.

ATTACK PATH

1. Confirm the payment POST has no anti-CSRF token.
2. Build an auto-submitting HTML form on a third-party page.
3. A logged-in victim visiting that page unknowingly submits a payment.

BUSINESS RISK

Any web page a customer visits while logged in can **move their money** — a phishing email doesn't even need a link to steal credentials, just a hidden auto-submit form.

12 Money Glitch

Intermediate 325 pts

The wire-transfer API computes $\text{balance} - \text{amount}$ without checking that amount is positive.

ATTACK PATH

1. Send a transfer request with a large negative amount.
2. Subtracting a negative number increases the balance instead of decreasing it.
3. Balance inflates arbitrarily on demand.

BUSINESS RISK

A missing sign check turns a debit endpoint into an **unlimited money-printing endpoint** — one of the highest-severity business-logic classes in fintech.

13 Bait and Switch

Intermediate 275 pts

The login flow's next= redirect parameter is trusted without validation.

ATTACK PATH

1. Craft a login link with
next=http://attacker-controlled-site.
2. Victim logs in normally, trusting the real bank domain.
3. Successful authentication silently redirects them to the attacker's look-alike page.

BUSINESS RISK

A trusted domain becomes a launchpad for **phishing and credential harvesting** — the exact mechanism behind many real "confirm your login" scam campaigns.

14 Password Roulette

Intermediate 350 pts

Password-reset tokens are a deterministic short hash of the username — no randomness, no expiry, no rate limit.

ATTACK PATH

1. Compute the token formula for a known username.
2. Use the predicted token directly against the reset endpoint.
3. Set a new password for someone else's account.

BUSINESS RISK

Full account takeover for any customer whose username is known or guessable — no phishing, no interaction with the victim required.

15 The Vault Manifest

Intermediate 300 pts

A data-team export endpoint returns every customer's PII and full card data — with no authentication at all.

ATTACK PATH

1. Request the export endpoint with no session or credentials.
2. Receive the full customer dataset in the response.
3. Locate the sensitive note embedded in one record.

BUSINESS RISK

This is a **notifiable mass data breach** in real life — full PII and card data for the entire customer base, reachable by anyone who finds the URL.

16 The Archivist

Intermediate 275 pts

The staff file archive strips literal `../` once — a well-known, trivially bypassed filter.

ATTACK PATH

1. Note the filter only removes one occurrence of `../`.
2. Submit `....//` — after the strip, it collapses back into `../`.
3. Walk out of the intended reports directory to read arbitrary server files.

BUSINESS RISK

A "sanitisation" step that isn't idempotent is not sanitisation — **arbitrary file read** on the server from a feature meant to serve quarterly PDFs.

19 The Fast Track

Intermediate 325 pts

The staff-onboarding API writes the entire client-supplied JSON body to the database — including the `role` field.

ATTACK PATH

1. Capture a normal onboarding request.
2. Add `"role": "admin"` to the JSON body.
3. The server trusts and persists a field it should set itself.

BUSINESS RISK

Mass assignment — any authenticated staff member can self-promote to administrator by editing a request body.

Tier 3 — Advanced

Attack chains

09 In the Dark

Advanced 350 pts

A credit-status lookup returns only "Approved" or "Not Found" — no errors, no data — but that boolean is enough.

ATTACK PATH

1. Confirm ' AND 1=1 -- vs ' AND 1=2 -- produce different responses.
2. Automate character-by-character extraction of a secrets-table value with an Intruder-style attack.
3. Submit the recovered value to unlock the vault.

BUSINESS RISK

Blind injection needs **no visible output** to be exploitable — silent failure modes are still catastrophic ones.

10 Document Confession

Advanced 375 pts

The payroll XML importer resolves external entities in uploaded documents.

ATTACK PATH

1. Submit a normal-looking payroll XML upload.
2. Replace it with a document declaring an external entity pointing at a local server file.
3. The parser inlines the file's contents into the rendered output.

BUSINESS RISK

Server-side file disclosure from a document-upload feature — a common entry point in back-office and HR tooling that rarely gets the same scrutiny as customer-facing code.

17 Internal Affairs

Advanced 425 pts

A staff maintenance console passes user input directly to a system shell.

ATTACK PATH

1. Gain staff access via an earlier chained bug.
2. Enter a normal-looking diagnostic value with a shell metacharacter appended.
3. The appended OS command executes with the server's privileges.

BUSINESS RISK

Remote code execution on the server itself — the ceiling outcome of a web vulnerability, with no further escalation needed.

18 Display Name Injection

Advanced 450 pts

A "live preview" of the profile display name renders user input as a server-side template rather than escaped data.

ATTACK PATH

1. Confirm template evaluation with an arithmetic probe like `{{7*7}}`.
2. Escalate to a payload that reaches the template engine's underlying OS module.
3. Execute an arbitrary command through the "preview" feature.

BUSINESS RISK

Server-side template injection → remote code execution from a feature as innocuous as a profile nickname field.

20 God Mode

Advanced 500 pts

A support "document fetcher" makes server-side requests on the player's behalf, and a companion config-import feature deserializes untrusted data — the platform's flagship multi-step chain.

ATTACK PATH

1. Confirm the obvious loopback address is blocklisted.
2. Bypass the blocklist with an alternate representation of localhost.
3. Reach an internal-only debug endpoint and retrieve a serialized payload.
4. Feed it to the config importer to complete the chain and extract the flag.

BUSINESS RISK

The highest point value on the board for a reason: **SSRF chained to insecure deserialization** is exactly the pattern behind several major cloud and enterprise breaches — a blocklist is not a security boundary.

08 — CONNECTIVITY & INFRASTRUCTURE RANGE

Beyond the app: a real network to operate in

The 20 challenges above all live inside vBank. Layered underneath them is an infrastructure-level exercise every player goes through regardless of which challenge they're on — the platform itself teaches practical connectivity and network literacy, not just application bugs.

Wildcard DNS & routing

A dedicated dnsmasq service resolves every *.lab hostname to the host machine; an nginx reverse proxy then routes each hostname to the correct container. Players experience — and implicitly learn — how subdomain-based routing and virtual hosting actually work.

TLS trust chain

The platform generates its own certificate authority and per-deployment certificates automatically. Players install one CA certificate, which is itself a practical lesson in how certificate trust and HTTPS interception work — the same trust model a proxy tool like Burp Suite depends on.

The Pwn Machine — a browser-delivered attacker workstation

Each player can launch a full Kali Linux desktop (noVNC, streamed to the browser) preloaded with nmap, Burp Suite, sqlmap, hydra, hashcat, wfuzz, gobuster, ffuf, tshark, masscan, jwt_tool, mitmproxy, and Python security tooling. No installation, no admin rights needed on the player's own machine — critical for participants on locked-down corporate laptops.

Per-player private network segment

Every player's vBank target and its analytics sidecar sit on their own isolated Docker bridge network (ctfd-net-{username}), reachable only via their assigned hostname/port — a live, personal, mini network to reconnoitre with nmap and traffic-analysis tools without ever touching another player's environment.

WHY THIS MATTERS BEYOND APPSEC

Participants leave with practical fluency in DNS, TLS/certificate trust, reverse proxying, and how to operate proxy/interception tooling against a live target — skills that generalise well beyond the specific web bugs in the catalog, and that most classroom-style training never touches.

09 — CONTAINMENT & SAFETY MODEL

Why it's safe to let people "hack a bank" at work

The single question every CISO asks first: what stops this from becoming a real incident? Four independent controls:

CONTROL	WHAT IT DOES
No shared blast radius	Every player gets their own vBank container, sidecar, and private Docker network. There is no shared account database, no shared filesystem, and one player's exploit cannot affect another's environment.
Fully synthetic data	Every account, balance, transaction, and "customer" is fabricated for the exercise. No real customer, employee, or financial data exists anywhere in the stack.
Network segmentation	The database and cache run on an internal-only Docker network with no route to players or the internet. Target containers are reachable only through the reverse proxy on the assigned LAN.
Automatic lifecycle limits	Every target and Pwn Machine self-destructs on a timer (default one hour, auto-extended only while actively used), and a background cleanup process reclaims idle infrastructure every 30 seconds.

The vulnerabilities in vBank are catalogued and intentional — see the internal engineering guardrail: they are explicitly documented as "do not fix," because fixing them would remove the exercise. This is the same operating model used by every reputable vulnerable-by-design training platform (e.g. DVWA, WebGoat, PortSwigger Web Security Academy), purpose-built for this organisation's own risk profile and workflows instead of a generic one.

— 10 — SCORING & PROGRESSION

How success is measured

- **Per-player scoring.** Flags are cryptographically derived per individual (HMAC of username), so the scoreboard reflects who actually did the work — flag-sharing does not work.
- **Point-weighted difficulty.** Point values run from 100 (read the page source) to 500 (the full SSRF-to-RCE chain), so the scoreboard doubles as a skill signal, not just a completion checklist.
- **Graduated hints.** Every challenge ships two hints at increasing point cost — nobody is stuck without a path forward, but full-price solves are rewarded.
- **Optional team layer.** Teams add visibility and camaraderie (join requests, captain approval, teammate-solve notifications) without changing individual accountability for flags or points.

11 — TECHNOLOGY STACK

Built on proven, auditable open-source foundations

LAYER	TECHNOLOGY
Competition engine	CTFd 3.8.6, extended with in-house plugins
Target application	Python / Flask (vBank), Node.js (analytics sidecar)
Attacker workstation	Kali Linux, XFCE desktop, noVNC — streamed to browser
Reverse proxy & routing	nginx-proxy, automatic TLS via a self-generated CA
DNS	dnsmasq, wildcard *.1ab resolution
Data & cache	MariaDB 10.11, Redis 7 (internal network only)
Orchestration	Docker Compose, on-demand per-player container lifecycle
Host platforms	Windows, macOS (incl. Apple Silicon via emulation), Linux

— 12 — SCALABILITY & DEPLOYMENT

From a single laptop to a company-wide event

The identical stack runs two ways, with no code changes required to move between them:

- **Single node** — one machine (Windows, macOS, or Linux) runs everything. Sufficient for a team offsite or a departmental exercise.
- **Four-node cluster** — a front-door HAProxy load balancer with sticky sessions distributes players across four backend hosts sharing one MariaDB and Redis instance, so scores, solves, and sessions stay consistent across the whole event regardless of which node a player lands on. Documented and tested for larger, company-wide exercises.

Typical single-node hardware requirement: 4+ CPU cores, 8GB+ RAM, 50GB disk — commodity hardware, no specialised infrastructure spend required to pilot the program.

13 — RECOMMENDATIONS

Suggested next steps

1. **Pilot with a single cohort.** Run a two-week, opt-in window for engineering and QA before mandating participation more broadly, to gather friction points and completion-rate data.
2. **Tie completion to onboarding.** New engineering hires complete the Basic and Intermediate tiers in their first month — a practical substitute for (or complement to) generic e-learning security modules.
3. **Quarterly refresh events.** A live, timed leaderboard event once a quarter sustains engagement better than a one-off rollout; the disposable, self-resetting infrastructure makes repeat events near-zero marginal effort.
4. **Feed findings back into secure-coding standards.** Each challenge maps to a concrete remediation pattern (parameterised queries, signed and verified tokens, allowlisted redirects, atomic transactions); use low completion rates per challenge as a signal for where internal secure-coding guidance needs to be stronger.
5. **Report participation as a board-level security metric.** Completion rate and score distribution across the organisation are concrete, auditable evidence of proactive security-culture investment.

OWASP Top 10 (2021) coverage map

OWASP CATEGORY	CHALLENGES
A01 — Broken access control	The Ghost Investor, Flagged for Review, No Token No Problem, Bait and Switch, The Vault Manifest, The Archivist
A02 — Cryptographic failures	The Repeating Pattern
A03 — Injection	All Doors Open, In the Dark, Feedback Loop, Internal Affairs, Display Name Injection
A04 — Insecure design	Double Dip, Money Glitch, The Fast Track
A05 — Security misconfiguration	The Intern's Mistake, Document Confession
A07 — Identification & authentication failures	Unsigned Agreement, Password Roulette
A08 / A10 — Software & data integrity failures / SSRF	God Mode

Glossary

- **IDOR** — Insecure Direct Object Reference: accessing another user's data by guessing or changing an identifier.
- **SSRF** — Server-Side Request Forgery: tricking a server into making requests to internal systems on the attacker's behalf.
- **SSTI** — Server-Side Template Injection: user input evaluated as executable template code rather than displayed as text.
- **TOCTOU** — Time-Of-Check to Time-Of-Use: a race condition where a security check and the action it protects happen far enough apart to be exploited.
- **XXE** — XML External Entity injection: an XML parser resolving external references to read local files or reach internal services.

- **Flag** — A unique, per-player proof string generated by successfully completing a challenge, submitted for points.

vBank Cyber Range — Executive Briefing · Internal, Confidential · Security Engineering · This document contains no live credentials, real customer data, or exploitable production information.